

Experiment No.	4
Experiment Title	Email Spam/Ham Classification using Logistic Regression and Support Vector Machines ¹
Student Name	Preethi D
Register Number	3122247001046
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02-08-26

1 Objective

To classify emails as spam or ham using Logistic Regression and Support Vector Machines (SVM) with four different kernels (linear, polynomial, RBF, sigmoid), and to compare the models on accuracy, F1-score, and training time before and after hyperparameter tuning.

2 Problem Statement

Given the Spambase dataset, where each email is represented by 57 numeric word/character-frequency and capital-run-length features, the objective is to predict a binary label indicating spam (1) or ham (0). Input: 57 numeric features per email. Output: binary class label. Logistic Regression and four SVM kernel variants are trained, tuned via GridSearchCV, and compared on accuracy, precision, recall, and F1-score.

3 Dataset Description

Dataset Name	Spambase (<code>spambase_csv.csv</code>)
Dataset Source	UCI Machine Learning Repository
Number of Samples	4,601
Number of Features	57 numeric features (word/character frequencies and capital-run-length statistics)
Number of Classes	2 (spam = 1, ham = 0)
Missing Values	0
Train-Test Split	80% train / 20% test (stratified), giving 3,680 training and 921 test samples

¹GitHub Repository: <https://github.com/preethi340/Machine-Learning-.git>

4 Exploratory Data Analysis

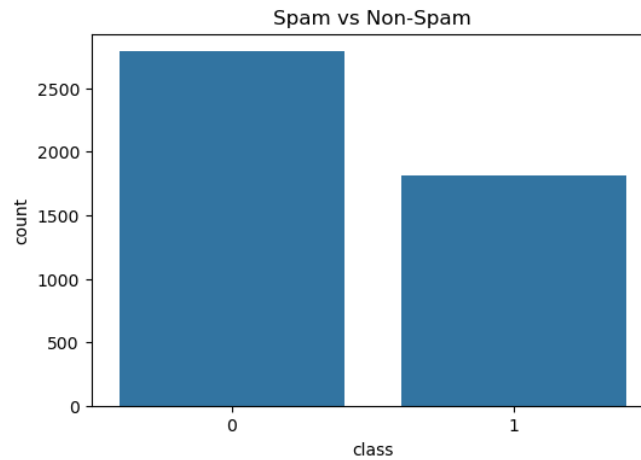


Figure 1: Class distribution: Spam vs. Non-Spam

Inference: The dataset is moderately imbalanced, with noticeably more non-spam (ham) emails than spam emails, roughly in a 3:2 ratio. This motivated the use of a stratified train-test split so that both classes are represented proportionally in training and testing, and it means that precision/recall/F1 are more informative than accuracy alone when comparing models.

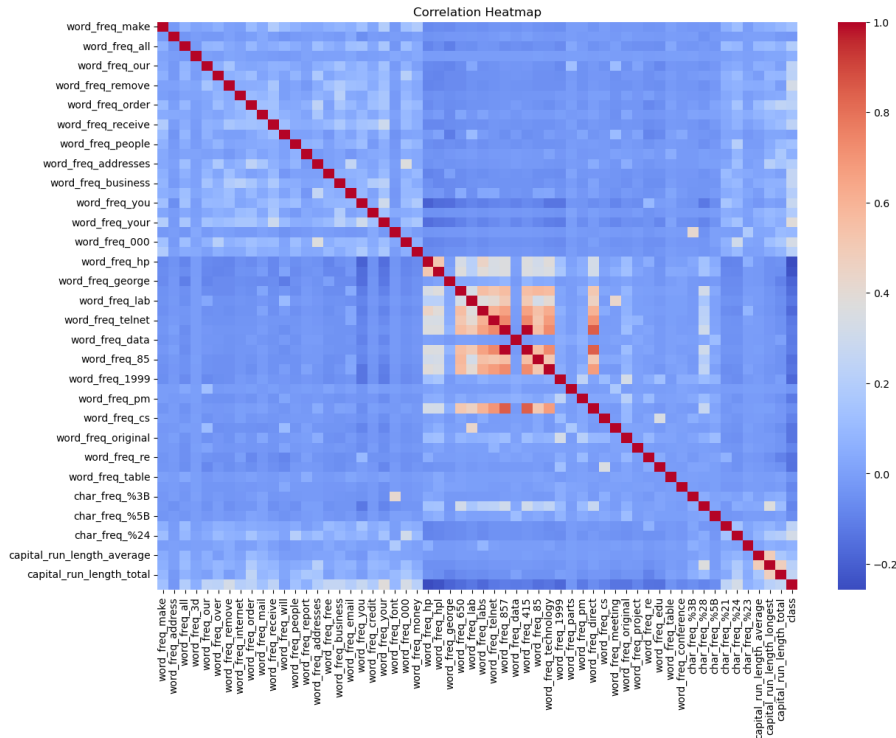


Figure 2: Correlation heatmap of all Spambase features

Inference: Most word-frequency features are weakly correlated with one another, since each represents a distinct, largely independent vocabulary term, though a handful

of related business/technology-themed words show mild positive correlation. This limited redundancy suggests most of the 57 features contribute at least some distinct information, supporting the decision to use all of them as model inputs without manual feature selection.

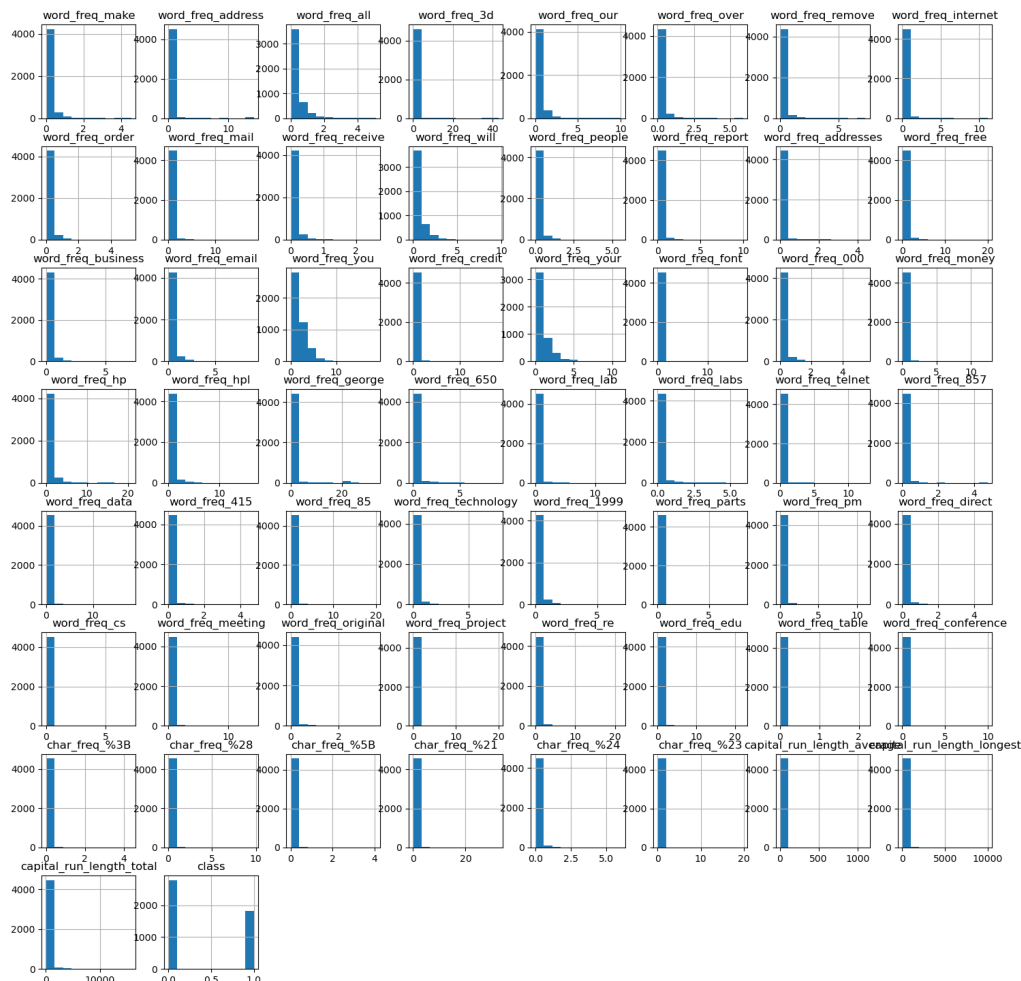


Figure 3: Distribution of all 58 dataset columns (features and target)

Inference: Nearly every word/character-frequency feature is extremely right-skewed, with most emails scoring 0 or near 0 and a small number of emails using that word/character heavily – a hallmark of sparse, bag-of-words-style data. This skew is part of why feature scaling (standardization) is applied before training either Logistic Regression or SVM, both of which are sensitive to feature scale.

5 Data Preprocessing

- **Missing value handling:** None required – the dataset has zero missing values across all 58 columns.
- **Encoding:** Not required, since all features and the target are already numeric.
- **Feature scaling:** All 57 numeric features were standardized using `StandardScaler` (zero mean, unit variance), fit on the training set and applied to both train and

test sets.

- **Train-test split:** An 80/20 stratified split was used (`train_test_split` with `stratify=y`, `random_state=42`), giving 3,680 training samples and 921 test samples while preserving the original class ratio.
- **Feature engineering:** None – all 57 original features were used directly as model inputs.

6 Machine Learning Algorithm

Logistic Regression:

- **Working principle:** Logistic Regression models the probability that a sample belongs to the positive class using a linear combination of features passed through the sigmoid function, and classifies based on a probability threshold (typically 0.5).
- **Mathematical formulation:** $P(y = 1 | x) = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}}$, with parameters learned by minimizing the regularized log-loss $-\sum_i [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)] + \lambda R(w)$, where $R(w)$ is an L_1 or L_2 penalty.
- **Advantages:** Fast to train, highly interpretable via feature coefficients, outputs well-calibrated probabilities, and works well when classes are close to linearly separable.
- **Limitations:** Assumes a roughly linear decision boundary in the feature space, so it may underperform when the true relationship between word frequencies and spam/ham is more non-linear.

Support Vector Machine (SVM):

- **Working principle:** SVM finds the hyperplane that maximizes the margin between the two classes; different kernel functions implicitly map the data into higher-dimensional spaces to capture non-linear boundaries: linear (no transformation), polynomial, radial basis function (RBF), and sigmoid.
- **Mathematical formulation:** The decision function is $f(x) = \sum_i \alpha_i y_i K(x_i, x) + b$, where K is the kernel function – e.g. RBF: $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$ – and α_i are learned by maximizing the margin subject to a soft-margin penalty controlled by C .
- **Advantages:** Effective in high-dimensional feature spaces (57 features here), the kernel trick allows capturing non-linear decision boundaries without explicitly computing the transformed features, and the RBF kernel in particular is very flexible.
- **Limitations:** Training and hyperparameter tuning (kernel, C , γ , degree) can be computationally expensive, performance is sensitive to the choice of kernel and its parameters (the polynomial kernel performed markedly worse than the others here), and SVMs do not naturally output calibrated class probabilities.

7 Experimental Setup

Python Version	
Scikit-Learn Version	
IDE	Jupyter Notebook
Operating System	Windows
Hardware	

8 Hyperparameter Selection

GridSearchCV – Logistic Regression (5-fold, scoring=accuracy):

Parameter	Value /
penalty searched	l1, l2
C searched	0.01, 0.1, 1, 10
solver searched	liblinear, lbfgs, newton-cg, newton-choi, sgd
Best penalty	l1
Best C	100
Best solver	liblinear
Best CV accuracy	0.9239

GridSearchCV – SVM (5-fold, scoring=accuracy):

Parameter	Value / Search Space
kernel searched	linear, poly, rbf, sigmoid
C searched	0.1, 1, 10, 100
gamma searched	scale, auto
degree searched	2, 3, 4
Best kernel	rbf
Best C	10
Best gamma	scale
Best degree	2
Best CV accuracy	0.9332

9 Results

Baseline models (default hyperparameters, test set of 921 samples):

Model	Accuracy	F1-score	Training Time (s)
Logistic Regression (default)	92.94%	0.9093	0.1730
Linear SVM	92.94%	0.9093	0.4622
Polynomial SVM	77.96%	0.6220	0.4604
RBF SVM	92.73%	0.9055	0.2857
Sigmoid SVM	88.49%	0.8528	0.2927

Tuned models (best estimator from GridSearchCV):

Model	Test Accuracy	Precision (spam)	Recall (spam)
Logistic Regression (tuned: C=100, l1, liblinear)	92.51%	0.92	0.89
SVM (tuned: rbf, C=10, gamma=scale, degree=2)	92.07%	0.91	0.88

5-Fold Cross-Validation (on training set, tuned models): Logistic Regression fold accuracies were 0.9402, 0.9185, 0.9144, 0.9198, 0.9266 (average **0.9239**); SVM fold accuracies were 0.9429, 0.9321, 0.9334, 0.9212, 0.9361 (average **0.9332**), showing SVM generalizes marginally better under cross-validation despite the two models being close on the single test split.

10 Result Visualization

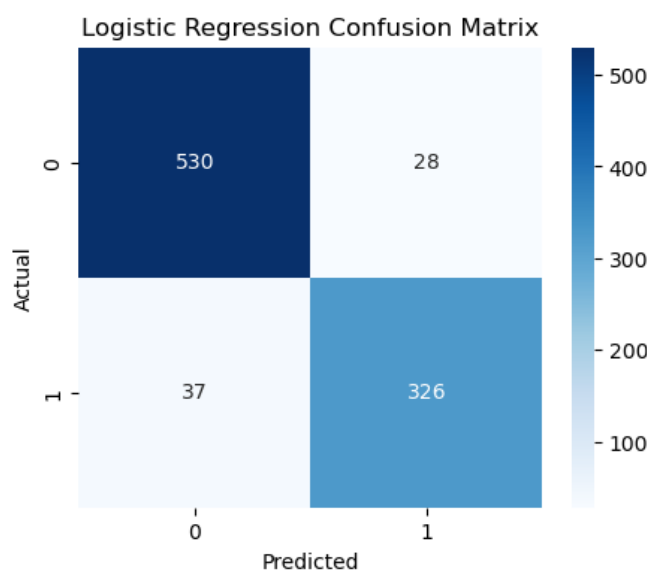


Figure 4: Confusion matrix – Logistic Regression (default, test set)

Inference: Logistic Regression correctly classifies the large majority of both spam and ham emails, with a modest number of false negatives (spam predicted as ham) slightly outnumbering false positives (ham predicted as spam). This mild asymmetry, and the correspondingly higher precision than recall for the spam class (0.92 vs. 0.90), suggests the model is a little conservative about labelling an email as spam.

Model	Accuracy	F1-score
Logistic Regression (tuned)	0.9207	0.8976
Linear SVM	0.9294	0.9093
Polynomial SVM	0.7796	0.6220
RBF SVM	0.9273	0.9055
Sigmoid SVM	0.8849	0.8528

Table 4: Final accuracy/F1 comparison across all five model configurations

Inference: The Linear SVM achieves the best raw accuracy and F1-score of all five configurations tested, narrowly ahead of RBF SVM and default Logistic Regression, while the Polynomial SVM is a clear outlier, performing far worse than every other model. This suggests that the true decision boundary between spam and ham, in this standardized 57-feature space, is close to linear, and that the polynomial kernel’s default settings are poorly suited to this dataset (consistent with the RBF kernel being selected as the GridSearchCV winner over polynomial).

11 Discussion

Logistic Regression and the Linear and RBF SVM kernels all achieved similar, strong performance (all above 92% accuracy), which suggests that spam vs. ham is close to linearly separable in this 57-dimensional standardized feature space. The Sigmoid kernel performed noticeably worse, and the Polynomial kernel performed far worse than all others (78% accuracy, 0.62 F1), likely because its default settings (degree 3, unscaled coefficients) do not suit this data well – consistent with GridSearchCV settling on `degree=2` when the polynomial kernel is included in a full search. Interestingly, after hyperparameter tuning, both Logistic Regression and the best SVM configuration produced very similar cross-validated accuracy (92.4% vs. 93.3%), with SVM holding a slight edge, though Logistic Regression remains attractive for its simplicity, speed, and interpretability. A useful next step would be to also standardize the polynomial kernel’s coefficient (`coef0`) during tuning, and to examine feature coefficients from the tuned Logistic Regression model to identify which specific words are most predictive of spam.

12 Conclusion

This experiment implemented and compared Logistic Regression and four SVM kernel variants for spam/ham classification on the Spambase dataset. The Linear SVM achieved the highest raw test accuracy (92.94%) and F1-score (0.9093) among all configurations, while GridSearchCV-tuned SVM (RBF kernel, $C=10$) achieved the best cross-validated accuracy (93.3%), narrowly ahead of tuned Logistic Regression (92.4%). The Polynomial kernel was clearly unsuitable for this dataset under default settings, underscoring the importance of kernel selection and hyperparameter tuning when applying SVMs.

13 References

1. M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, “Spambase Data Set,” UCI Machine Learning Repository, 1999.
2. F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
3. C. Cortes and V. Vapnik, “Support-Vector Networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
4. D. W. Hosmer, S. Lemeshow, and R. X. Sturdivant, *Applied Logistic Regression*, 3rd ed. Wiley, 2013.